

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.
(BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks: 20

Annexure A

Descriptive Written Test

Code of Conduct:

I Athavan K(192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Athavan K

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor understanding, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Question 1 — Types of Intelligent Agents in a Smart Home Automation System

Problem Context:

The smart home system controls lighting, temperature, and security. It must react instantly to real-time percepts (motion, temperature change, time of day) and also learn user preferences over time. This dual requirement — fast reactive behaviour plus adaptive long-term optimisation — makes it an ideal case study for comparing the four classical agent architectures. Before analysing each agent type, it helps to fix the system's PEAS description:

PEAS Element	Smart Home Description
Performance measure	Occupant comfort, energy savings, security (intrusion prevention), and minimal false alarms/adjustments.
Environment	Rooms of a house with occupants, ambient temperature, daylight/time-of-day cycle, doors/windows.
Actuators	Light switches/dimmers, HVAC (heating/cooling) unit, door locks, alarm system, notifications.
Sensors	Motion (PIR) sensors, temperature sensors, light sensors, door/window contact sensors, clock/calendar, cameras.

1.1 Simple Reflex Agent

A simple reflex agent selects actions using only the current percept, matched against condition–action rules of the form 'if condition then action'. It has no memory of the past and no model of unobserved aspects of the world.

- Functioning in the smart home: 'IF motion detected in a room THEN turn on the light'; 'IF temperature > 28°C THEN turn on the AC'; 'IF door opened after 11 PM AND no scheduled occupant THEN trigger alarm'.
- Strength: extremely fast, low computational cost, ideal for safety-critical, instantaneous responses (e.g., turning on a light the instant motion is sensed, or sounding an intrusion alarm).
- Weakness: cannot distinguish context — it would turn lights on for a pet's motion at 3 AM exactly as it would for a person, cannot remember that a room was already lit 10 seconds ago, and cannot adapt to schedules or preferences. It works only in a fully observable, simple environment.

1.2 Model-Based Reflex Agent

A model-based agent maintains an internal state that tracks aspects of the world it cannot currently sense, updated using a model of how the world evolves and how its own actions affect it.

- Functioning in the smart home: it keeps an internal record such as 'living room has been unoccupied for 15 minutes', 'it is currently night-time', 'the last known indoor temperature trend is rising', or 'family members X and Y are away at work'. This lets it decide, for example, to dim rather than switch off a light when a room is briefly unoccupied, or to pre-cool a room before someone is expected home, even though no one is present to sense right now.
- Strength: handles partial observability far better than a simple reflex agent — it can reason about rooms or times it is not currently sensing.
- Weakness: still rule-driven; it does not explicitly plan ahead to satisfy a goal, and it has no way to weigh trade-offs between competing objectives such as comfort versus energy cost.

1.3 Goal-Based Agent

A goal-based agent has explicit goals and chooses actions (possibly by searching/planning ahead) that it predicts will lead to a goal state, rather than reacting to a fixed rule table.

- Functioning in the smart home: given the goal 'maintain bedroom temperature at 22°C by 10 PM', the agent can reason about how long the HVAC needs to run given the current temperature and outside conditions, and plan when to start it. Given the goal 'house is secure before occupants sleep', it can plan a sequence of actions — lock all doors, arm the alarm, turn off non-essential lights — rather than reacting to each event independently.
- Strength: far more flexible than reflex-based designs — a new goal (e.g., 'prepare the house for a guest arriving at 6 PM') can be handled without rewriting condition–action rules.
- Weakness: goals are typically boolean (achieved or not achieved); it does not naturally express 'how good' an outcome is when several goals conflict (e.g., comfort vs energy saving), and planning can be computationally heavier than reflex rule matching.

1.4 Utility-Based Agent

A utility-based agent extends the goal-based agent by using a utility function that assigns a numeric 'desirability' score to each possible state, allowing it to choose the action that maximises expected utility when multiple, sometimes conflicting, goals are in play.

- Functioning in the smart home: a utility function can combine `comfort_score(temperature, lighting)`, minus `energy_cost(HVAC runtime, lighting duration)`, plus `security_score(doors locked, alarm armed)` into a single weighted

objective. The agent then picks the setting — e.g., 23°C instead of the 'ideal' 21°C — that best balances comfort against electricity cost, or decides whether the marginal comfort gain of extra cooling is worth the marginal energy expense.

- **Strength:** the only architecture among the four that can natively resolve trade-offs between competing objectives (comfort, energy efficiency, security) instead of treating them as independent yes/no rules or goals.
- **Weakness:** requires a carefully designed (and ideally learned) utility function; a poorly weighted function can produce unintuitive behaviour, and computing/optimising utility across many variables is more expensive than simple rule matching.

1.5 Comparative Summary

Agent Type	Uses Internal State?	Handles Multiple Competing Goals?	Smart-Home Example	Best Suited For
Simple Reflex	No	No	Motion → light ON	Instant safety/security reactions
Model-Based Reflex	Yes (world model)	No (still rule-based)	Dim light after room is inferred empty	Partial observability handling
Goal-Based	Yes + explicit goals	Only sequentially, not weighted	Plan pre-cooling before target time	Achieving a specific defined objective
Utility-Based	Yes + utility function	Yes (weighted trade-offs)	Balance comfort vs energy cost vs security risk	Optimising across conflicting objectives

1.6 The Learning Dimension

The requirement that the system 'learns user preferences over time' points to a Learning Agent architecture layered on top of the above — consisting of a performance element (the actual agent, e.g., the utility-based decision-maker described above), a critic that evaluates outcomes against feedback (e.g., the user manually overriding a temperature setting), a learning element that adjusts the utility function or rules based on that feedback, and a problem generator that occasionally tries exploratory actions to gather more information (e.g., testing a slightly different schedule). Over time this lets the utility

function's weights — and even the model-based agent's assumptions about occupancy patterns — improve from experience rather than being hand-coded once.

1.7 Justification: Most Effective Design

No single classical agent type alone satisfies all three requirements — comfort, energy efficiency, and security — equally well, so a hybrid, layered architecture is justified:

- A simple reflex layer should handle safety-critical, latency-sensitive events (intrusion detection, fire/smoke response) — these must never be delayed by planning or utility computation.
- A model-based reflex layer should maintain the running internal state (occupancy history, time-of-day, recent temperature trend) that all higher layers depend on, since large parts of the house are not sensed at every instant.
- A utility-based layer, built on top of the model-based state, should make the routine, non-urgent decisions (HVAC scheduling, lighting levels, energy-vs-comfort trade-offs) because it is the only architecture that can quantitatively balance comfort against energy efficiency rather than treating them as independent switches.
- A learning element wrapped around the utility-based layer should continuously refine the utility weights and the occupancy/preference model from user feedback and behaviour, satisfying the explicit 'learns user preferences over time' requirement.

In short: a Learning, Utility-Based agent — with an embedded Simple-Reflex safety override and a Model-Based state layer beneath it — is the most effective overall design for this smart home system, since it combines instant safety response, awareness of unobserved parts of the house, principled trade-off optimisation between comfort and energy, and continuous adaptation to the specific household.

Question 2 — Uninformed Search for an Unknown Maze (UCS and IDS)

Formulating the Maze as a Search Problem:

Component	Definition for the Maze Robot
State	The robot's current cell/position (and orientation, if turning has a cost) within the maze.
Initial state	The robot's starting cell.
Actions	Move Up / Down / Left / Right (or Forward/Turn) to an adjacent open cell.
Transition model	Moving from one cell to an adjacent, non-wall cell.
Goal test	Current cell = the maze exit.
Path cost	Number of moves, or a weighted cost per move if some passages (e.g., narrow/rough terrain) cost more than others.

2.1 Uniform Cost Search (UCS)

UCS is an uninformed search that always expands the frontier node with the lowest cumulative path cost $g(n)$ so far, using a priority queue ordered by $g(n)$ (it is Dijkstra's algorithm applied to a search tree/graph).

- How it helps in the maze: if different passages have different traversal costs (e.g., a muddy patch costs more to cross than an open corridor, or diagonal vs straight moves cost differently), UCS is guaranteed to find the cheapest path to the exit, not merely the path with fewest steps.
- It requires no knowledge of the maze layout in advance — cells and their costs are discovered as they are expanded — which matches the 'unknown maze' condition.
- Completeness/Optimality: UCS is complete and optimal as long as all step costs are positive (no negative-cost passages and costs are bounded away from zero), regardless of the maze's branching factor or depth.

2.2 Iterative Deepening Search (IDS)

IDS repeatedly performs a depth-limited depth-first search, increasing the depth limit by one each iteration (limit = 0, 1, 2, ...) until the goal is found. It combines DFS's low memory footprint with BFS's completeness and (for uniform step costs) optimality.

- How it helps in the maze: since the robot does not know the maze's size or the exit's depth in advance, IDS avoids having to guess a depth bound — it simply keeps deepening until the exit is found, so it is guaranteed to find the shallowest (fewest-move) exit path without ever exceeding the true solution depth by more than one extra iteration.

- Because it explores like DFS within each iteration, it only ever needs to store a single path from the start to the current frontier node, which is ideal for a robot with very limited onboard memory.

2.3 Advantages and Disadvantages — Time and Space Complexity

Algorithm	Time Complexity	Space Complexity	Advantages	Disadvantages
UCS	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$ — exponential in the depth of the optimal solution (b = branching factor, C^* = optimal cost, ϵ = minimum step cost)	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$ — must keep the entire frontier and explored set in memory (same order as BFS)	Guarantees the cheapest path even with varying step costs; works without a heuristic; complete for positive costs	High memory usage — impractical for large/deep mazes; can explore many low-cost-but-irrelevant nodes before reaching goal if costs vary widely
IDS	$O(b^d)$ — same asymptotic order as BFS/DFS (d = depth of shallowest goal), despite repeating shallow levels multiple times	$O(b \cdot d)$ — linear in depth, since only one root-to-frontier path is stored at a time	Very low memory footprint — practical for embedded/low-memory robots; complete; optimal when all step costs are equal (e.g., unweighted maze moves); does not need to know maze depth in advance	Repeated re-expansion of shallow nodes on every iteration wastes some computation; not optimal if step costs are unequal (it minimises number of moves, not total cost)

2.4 Relating the Problem to Intelligent Agent Types

- **Simple reflex agent:** cannot solve an unknown maze reliably — with no memory of visited cells it can loop indefinitely between the same two cells and has no way to recognise it is retracing its path.
- **Model-based reflex agent:** maintains an internal map of visited cells and known walls as it explores, which prevents infinite loops and lets it recognise dead ends

— a prerequisite for any systematic maze search, but by itself it still does not guarantee finding the exit efficiently or optimally.

- **Goal-based agent:** explicitly represents the goal ('reach the exit cell') and uses a search algorithm — precisely UCS or IDS — over its internal map to construct a full action sequence that provably achieves that goal, rather than just reacting cell by cell.
- **Utility-based agent:** becomes relevant if the maze has multiple exits or paths with different trade-offs (distance vs battery cost vs risk of rough terrain) — it would use a utility function over path cost, time, and risk to choose the best route among several found, rather than just the first or cheapest one found by a single metric.

In all cases, the underlying search strategy (UCS or IDS) is what the goal-based (or utility-based) agent's decision-making procedure actually runs internally; the agent architecture determines what information is available and what objective is optimised, while the search algorithm determines how the optimal or satisfactory action sequence is computed from that information.

2.5 How Agent Design Affects Decision-Making

- An agent's architecture bounds what search strategies are even meaningful for it: a simple reflex agent has no memory to run UCS/IDS at all (both algorithms inherently require storing explored/frontier state), so only model-based-or-higher agents can use them.
- Memory constraints of the agent (e.g., a small embedded robot) directly favour IDS over UCS, since IDS needs only $O(b \cdot d)$ memory versus UCS's exponential frontier storage — an agent designed with tight memory resources should be paired with IDS.
- If the agent's performance measure explicitly includes movement/energy cost (a utility-based objective) rather than just 'reach the goal', the agent's design requires an algorithm sensitive to path cost — UCS — since IDS alone only minimises the number of moves, not total weighted cost.
- A goal-based agent that only cares about reachability (binary goal test) is naturally matched to IDS: it needs completeness and optimal move-count, not cost-optimality.

2.6 Recommended Combination and Justification

Recommended: a Model-Based, Goal-Based agent using Iterative Deepening Search as the default strategy, with the ability to fall back to Uniform Cost Search when the maze has variable movement costs.

- The maze is unknown and its depth/size is not known in advance, and a physical maze-solving robot typically has limited onboard memory — this combination of conditions directly favours IDS's $O(b \cdot d)$ space complexity over UCS's exponential

frontier, while still guaranteeing a complete and (for uniform move costs) optimal solution.

- The agent must be at least model-based (maintaining a map of visited cells/walls) for either search strategy to terminate correctly and avoid redundant exploration — a simple reflex design is not viable here.
- It must be goal-based because the task is defined by a specific goal condition ('reached the exit'), not a utility trade-off, for the basic version of the problem — this keeps the search formulation simple (goal test rather than utility maximisation).
- If the maze's passages have unequal traversal costs (rough terrain, longer corridors, energy-expensive turns), the design should switch to UCS despite its higher memory cost, since only UCS guarantees the cheapest — not merely the shortest — path; in that case the agent effectively becomes utility-based, with 'utility' equal to negative path cost.
- Overall justification: pairing a model-based, goal-based agent with IDS gives the best balance of completeness, optimality (in moves), and practical memory efficiency for an unknown, uniform-cost maze — the single most common and resource-constrained version of this problem — while explicitly reserving UCS for the special case where path cost, not path length, is what must be minimised.